ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC HÀNH

ĐIỀU KHIỂN ROBOT DI ĐỘNG TURTLEBOT 4

BÀI 04: XÂY DỰNG BẢN ĐỒ SLAM, ĐỊNH VỊ LOCALIZATION VÀ ĐIỀU HƯỚNG TỰ HÀNH VỚI NAVIGATION 2 (NAV2)

Sinh viên thực hiện: Nguyễn Quang Tuân – 23020767

Giảng viên hướng dẫn: PGS.TS. Hoàng Văn Xiêm
CN. Nguyễn Quốc Bảo

Hà Nội – 2026

Mục lục

1 TỔNG QUAN HỆ THỐNG VÀ KIẾN TRÚC ĐIỀU KHIỂN 2
1.1 Cấu trúc thư mục dự án . . . 2
1.2 Kiến trúc của hệ thống . . . 2
1.3 Vai trò chi tiết của từng khối thành phần . . . 3
1.3.1 Khối Mô phỏng Vật lý (Gazebo Fortress & ros_gz_bridge) . . . 3
1.3.2 Khối Hỗ trợ Truyền thông & TF Relay (`tf_relay.py`) . . . 4
1.3.3 Khối Xây dựng Bản đồ (`slam_toolbox`) . . . 4
1.3.4 Khối Định vị Robot (`amcl` & `map_server`) . . . 4
1.3.5 Khối Điều hướng Tự hành Nav2 (`nav2_bringup`) . . . 4

2 CÁC PACKAGE, NODE VÀ HỆ THỐNG TOPIC / TF CỐT LÕI 5
2.1 Danh sách Package cốt lõi . . . 5
2.2 Danh sách Node chính và vai trò vận hành . . . 5
2.3 Phân tích các Topic cốt lõi và luồng dữ liệu . . . 6
2.4 Cây toạ độ không gian (TF Tree Architecture) . . . 6

3 QUY TRÌNH XÂY DỰNG BẢN ĐỒ VỚI SLAM TOOLBOX 8
3.1 Thiết lập môi trường và kịch bản thực nghiệm . . . 8
3.2 Lưu đồ thuật toán SLAM Toolbox . . . 8
3.3 Nguyên lý Graph-based SLAM, Scan Matching và Ceres Optimization . . . 9
3.4 Cấu trúc file dữ liệu bản đồ hoàn chỉnh (`map.yaml` & `map.pgm`) . . . 9

4 QUY TRÌNH ĐỊNH VỊ ROBOT VỚI AMCL (LOCALIZATION) 10
4.1 Thiết lập môi trường và quy trình nạp bản đồ tĩnh . . . 10
4.2 Nguyên lý bộ lọc hạt thích nghi Monte Carlo (AMCL) . . . 10
4.3 Khởi tạo Initial Pose, No-motion Update và cơ chế hội tụ hạt . . . 11

5 CÁCH NAV2 LẬP KẾ HOẠCH VÀ ĐIỀU KHIỂN TURTLEBOT 4 12
5.1 Cấu trúc phân lớp của ngăn xếp Navigation 2 (Nav2 Stack) . . . 12
5.2 Bản đồ chi phí đa tầng (Costmaps) và cơ chế Inflation Layer . . . 12
5.3 Bộ lập kế hoạch toàn cục (Global Planner – NavFn) . . . 12
5.4 Bộ điều khiển bám quỹ đạo cục bộ (Local Controller – DWB) . . . 12
5.5 Cây hành vi và cơ chế tự phục hồi (Recovery Behaviors) . . . 13

6 TRẢ LỜI CÁC CÂU HỎI TRONG PHẦN "NỘI DUNG CẦN HIỂU" 14
6.1 Câu 1: Tại sao phải kết hợp cả Odometry và LiDAR trong SLAM và AMCL mà không chỉ dùng một loại cảm biến duy nhất? . . . 14
6.2 Câu 2: Sự khác biệt cốt lõi giữa hệ toạ độ `map` và `odom`? Tại sao transform `map` → `odom` có thể nhảy cóc, còn `odom` → `base_link` bắt buộc phải biến thiên liên tục mượt mà? . . . 14
6.3 Câu 3: Vai trò của Inflation Layer trong Costmap? Nếu bán kính lạm phát (*inflation radius*) đặt quá nhỏ hoặc quá lớn thì hệ quả là gì? . . . 15
6.4 Câu 4: Cơ chế hoạt động của Recovery Behaviors trong Navigation 2? Nêu thứ tự kích hoạt điển hình khi robot bị kẹt. . . . 15
6.5 Câu 5: Ưu và nhược điểm của Synchronous SLAM so với Asynchronous SLAM trong SLAM Toolbox? . . . 16

7 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG 17
7.1 Kết quả kiểm tra Simulation và Topic tự động (`check_topics`) . . . 17
7.2 Kết quả xây dựng bản đồ trên RViz2 và bản đồ lưới hoàn chỉnh . . . 17
7.3 Kết quả Localization và hội tụ đám mây hạt AMCL . . . 18

7.4 Kết quả thực hiện 03 Navigation Goals liên tiếp 18
7.5 Log terminal thực tế của quá trình thực thi Auto Nav Goals 18

8 CẢI TIẾN KỸ THUẬT VÀ TỐI ƯU HÓA HỆ THỐNG 20
8.1 Khắc phục hiện tượng Deadlock bộ nhớ chia sẻ với FastDDS UDP-only 20
8.2 Khắc phục lỗi Transform Extrapolation bằng Node TF Relay 20
8.3 Tự động hóa hoàn toàn chu trình: Auto Undock → Pose → Nav Goals 20

9 ĐƯỜNG DẪN DỰ ÁN VÀ TÀI NGUYÊN 21
9.1 Tập lệnh khởi động nhanh (Quick Start Guide) 21

1 TỔNG QUAN HỆ THỐNG VÀ KIẾN TRÚC ĐIỀU KHIỂN

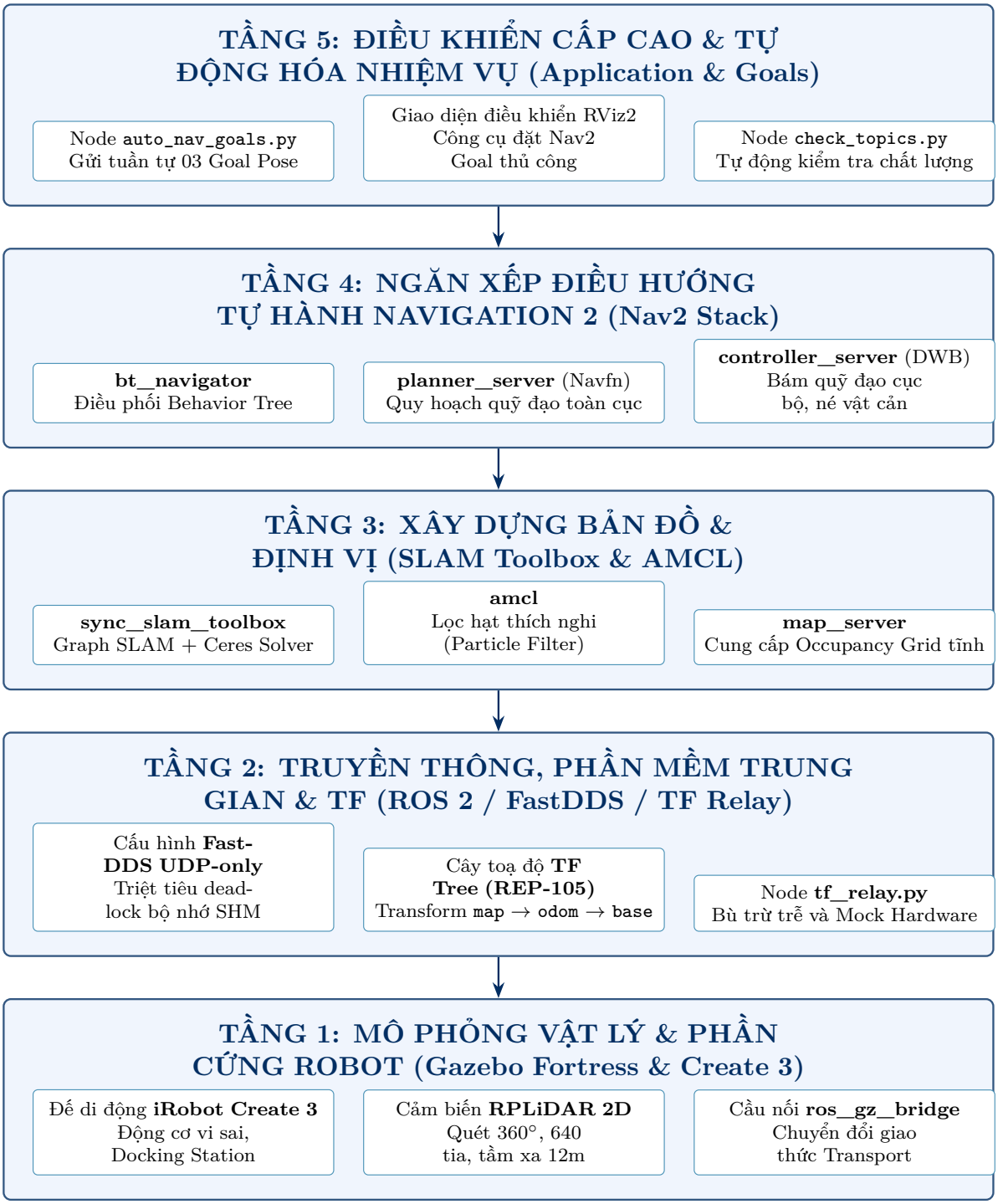
1.1 Cấu trúc thư mục dự án

Toàn bộ gói chương trình được xây dựng theo chuẩn ROS 2 Ament Python đặt trực tiếp bên trong thư mục `src/turtlebot4_practice4` của workspace `turtlebot4_ws`. Hệ thống vận hành hoàn chỉnh trên nền tảng ROS 2 Humble Hawksbill, hệ điều hành Ubuntu 22.04 LTS kết hợp môi trường mô phỏng vật lý Gazebo Fortress (Ignition Gazebo), trực quan hóa dữ liệu qua RViz2 và các ngăn xếp thuật toán SLAM Toolbox, AMCL, Nav2:

```
turtlebot4_ws/
|-- src/
|   |-- turtlebot4_practice4/            # Package thuc hanh chinh
|   |   |-- CMakeLists.txt / setup.py    # Cau hinh package va console_scripts
|   |   |-- package.xml                  # Tuyen bo phu thuoc goi ROS 2
|   |   |-- turtlebot4_practice4/
|   |   |   |-- check_topics.py          # Kiem tra tu dong /scan, /odom, /cmd_vel
|   |   |   |-- auto_undock.py           # Tu dong go dock roi khoi tram sac
|   |   |   |-- tf_relay.py              # Dong bo transform va mock service
|   |   |   `-- auto_nav_goals.py        # Tu dong thuc thi 03 Navigation Goals
|   |   |-- launch/
|   |   |   |-- sim_robot.launch.py      # Launch mo phong Gazebo + TurtleBot 4
|   |   |   |-- sim_slam.launch.py       # Launch SLAM Toolbox + RViz2
|   |   |   |-- sim_localization.launch.py # Launch AMCL + Map Server
|   |   |   |-- sim_navigation.launch.py # Launch toan bo ngan xep Nav2
|   |   |   `-- view_rviz.launch.py      # Launch giao dien truc quan hoa RViz2
|   |   |-- config/
|   |   |   |-- slam_toolbox_params.yaml # Tham so toi uu cho SLAM Toolbox
|   |   |   |-- localization_params.yaml # Tham so bo loc hat AMCL
|   |   |   |-- nav2_params.yaml         # Costmaps, DWB Controller, NavFn Planner
|   |   |   |-- fastdds_udp_only.xml     # Cau hinh FastDDS loai bo loi deadlock SHM
|   |   |   |-- turtlebot4_mapping.rviz  # Giao dien RViz2 Mapping
|   |   |   `-- turtlebot4_nav2.rviz     # Giao dien RViz2 Navigation
|   |   |-- maps/
|   |   |   |-- map.yaml / map.pgm       # Ban do Occupancy Grid hoan chinh
|   |   |   |-- warehouse_map.yaml/.pgm  # Ban do moi truong warehouse
|   |   |   `-- maze_map.yaml/.pgm       # Ban do moi truong maze
|   |   |-- scripts/
|   |   |   |-- run_part1_sim.sh         # Tap lenh chay Phan 1: Simulation
|   |   |   |-- run_part2_slam.sh        # Tap lenh chay Phan 2: SLAM Mapping
|   |   |   |-- save_map.sh              # Tap lenh luu ban do tu SLAM
|   |   |   |-- run_part3_localization.sh # Tap lenh chay Phan 3: Localization
|   |   |   |-- run_part4_navigation.sh  # Tap lenh chay Phan 4: Nav2
|   |   |   `-- run_auto_goals.sh        # Tap lenh tu dong 03 Goals
|   |   `-- docs/                        # Tai lieu va bao cao ky thuat
|   `-- irobot_create_control/           # Dieu khien dong hoc vi sai Create 3
|-- build/
`-- install/
```

1.2 Kiến trúc của hệ thống

Hệ thống được thiết kế theo mô hình phân tầng 5 lớp độc lập (*Hierarchical 5-Layer Architecture*), bảo đảm nguyên tắc phân tách trách nhiệm (*Separation of Concerns*) và tính tương thích cao giữa phần cứng mô phỏng và các thuật toán tự hành thông minh:



Hình 1: Kiến trúc phân tầng 5 lớp của hệ thống điều khiển tự hành robot TurtleBot 4

1.3 Vai trò chi tiết của từng khối thành phần

1.3.1 Khối Mô phỏng Vật lý (Gazebo Fortress & ros_gz_bridge)

- Đảm nhiệm mô phỏng chính xác động học bánh xe vi sai (*differential drive kinematics*), tương tác ma sát tiếp xúc giữa bánh xe và mặt sàn nhà kho (*warehouse*).
- Mô phỏng cảm biến quét Laser RPLiDAR 2D với 640 tia quét, cự ly đo kiểm từ 0.15 m đến 12.0 m và tần số lấy mẫu 10 Hz.
- Gói cầu nối `ros_gz_bridge` thực hiện biên dịch hai chiều các thông điệp giữa Ignition Transport và ROS 2 DDS.

1.3.2 Khối Hỗ trợ Truyền thông & TF Relay (`tf_relay.py`)

- Xử lý đặc tính trễ thời gian trong môi trường mô phỏng bằng cách đồng bộ hoá simulation clock và đảm bảo cây toạ độ TF luôn liên tục.
- Cung cấp các dummy service phần cứng (`start_motor`, `oakd/start_camera`...) vốn có trên TurtleBot 4 vật lý nhằm ngăn chặn các cảnh báo lỗi treo tiến trình trong quá trình khởi tạo node.

1.3.3 Khối Xây dựng Bản đồ (`slam_toolbox`)

- Hoạt động ở chế độ `sync_slam_toolbox_node`, thu nhận dữ liệu quét laser và odometry để xây dựng đồ thị vị thế (*Pose Graph*).
- Ứng dụng bộ giải phi tuyến Ceres Solver tối ưu hóa vị trí các điểm mốc và kích hoạt thuật toán đóng vòng lặp (*Loop Closure*) nhằm triệt tiêu hoàn toàn sai số trôi dạt tích lũy của bánh xe.

1.3.4 Khối Định vị Robot (`amcl` & `map_server`)

- `map_server` nạp bản đồ tĩnh dạng lưới chiếm dụng (*Occupancy Grid*) từ tệp `map.yaml` và `map.pgm`.
- `amcl` sử dụng thuật toán bộ lọc hạt thích nghi Monte Carlo (*Adaptive Monte Carlo Localization*) với KLD-sampling để ước lượng chính xác tư thế robot (x, y, θ) trên hệ toạ độ toàn cục `map`.

1.3.5 Khối Điều hướng Tự hành Nav2 (`nav2_bringup`)

- Quản lý toàn diện chu trình tự hành khép kín: Bản đồ chi phí đa tầng (*Global & Local Costmaps*), thuật toán tìm đường Dijkstra/A* (*NavFn Planner*), bộ điều khiển bám quỹ đạo Dynamic Window Approach (*DWB Local Controller*).
- Cây hành vi (*Behavior Tree*) điều phối các hành động phục hồi (*Recovery Behaviors*) như làm sạch costmap, quay xe tại chỗ hoặc lùi xe khi gặp chướng ngại vật.

2 CÁC PACKAGE, NODE VÀ HỆ THỐNG TOPIC / TF CỐT LÕI

2.1 Danh sách Package cốt lõi

Tên Package	Vai trò chức năng trong hệ thống
`turtlebot4_ignition_bringup`	Khởi chạy môi trường mô phỏng Gazebo Fortress, nạp bản đồ thế giới (`warehouse`), spawn mô hình robot TurtleBot 4 và trạm sạc Dock.
`ros_gz_bridge`	Cầu nối trung chuyển dữ liệu hai chiều giữa Ignition Gazebo Transport và các Topic/Service của ROS 2.
`slam_toolbox`	Ngăn xếp SLAM 2D hiện đại dựa trên đồ thị tối ưu (Graph-based SLAM), xây dựng bản đồ thời gian thực từ dữ liệu Laser và Odometry.
`nav2_bringup` / `nav2_*`	Toàn bộ ngăn xếp Navigation 2: Quản lý vòng đời (Lifecycle), Costmaps, Planner Server, Controller Server và Behavior Tree Navigator.
`turtlebot4_navigation`	Cung cấp cấu hình khởi chạy mặc định cho các bài toán định vị và dẫn đường trên dòng robot TurtleBot 4.
`teleop_twist_keyboard`	Nhận lệnh điều khiển từ bàn phím và phát tín hiệu vận tốc `geometry_msgs/msg/Twist`.
`turtlebot4_practice4`	Gói thực hành tích hợp chứa toàn bộ Launch files, Configs tối ưu hóa, Scripts điều khiển tự động và Tài liệu báo cáo.

Bảng 1: Danh sách các package ROS 2 cốt lõi trong hệ thống

2.2 Danh sách Node chính và vai trò vận hành

- **`ign gazebo` / `ros_ign_bridge`**: Mô phỏng môi trường vật lý thế giới thực, mô hình va chạm, động cơ bánh xe và chuyển đổi cảm biến quang học sang các topic ROS 2 chuẩn.
- **`diffdrive_controller`**: Bộ điều khiển động học vi sai tiếp nhận lệnh vận tốc góc/dài từ topic `/cmd_vel` và chuyển thành vận tốc góc quay của các khớp bánh xe trái và phải.
- **`sync_slam_toolbox_node`**: Node SLAM đồng bộ hóa, nhận dữ liệu quét laser `/scan` và biến đổi `odom` để xây dựng lưới bản đồ và tối ưu đồ thị vị thế thông qua Ceres Solver.
- **`map_server`**: Nạp tệp siêu dữ liệu `map.yaml` cùng tệp ảnh `map.pgm` và phát liên tục bản đồ tĩnh lên topic `/map` với chất lượng dịch vụ Transient Local.
- **`amcl`**: Bộ định vị Monte Carlo thích nghi, theo dõi và ước lượng tư thế robot (x, y, θ) thông qua tập hạt xác suất `/particlecloud` và xuất ra biến đổi toạ độ `map -> odom`.
- **`planner_server`**: Tính toán đường đi không va chạm toàn cục từ điểm xuất phát tới đích dựa trên bản đồ chi phí toàn cục (*Global Costmap*) bằng thuật toán Dijkstra.
- **`controller_server`**: Tính toán lệnh vận tốc tức thời `/cmd_vel` ở tần số 20 Hz bám theo đường đi toàn cục và tránh va chạm vật cản tức thời qua thuật toán DWB.
- **`bt_navigator`**: Điều phối toàn bộ vòng đời nhiệm vụ dẫn đường qua cây hành vi (*Behavior Tree*), xử lý trạng thái lỗi và kích hoạt các hành vi phục hồi.
- **`auto_nav_goals`**: Node tự động hoá quy trình: Thực hiện Undock ra khỏi trạm sạc, khởi tạo Initial Pose, kiểm tra các node sẵn sàng và điều hướng robot qua 03 mục tiêu liên tiếp.

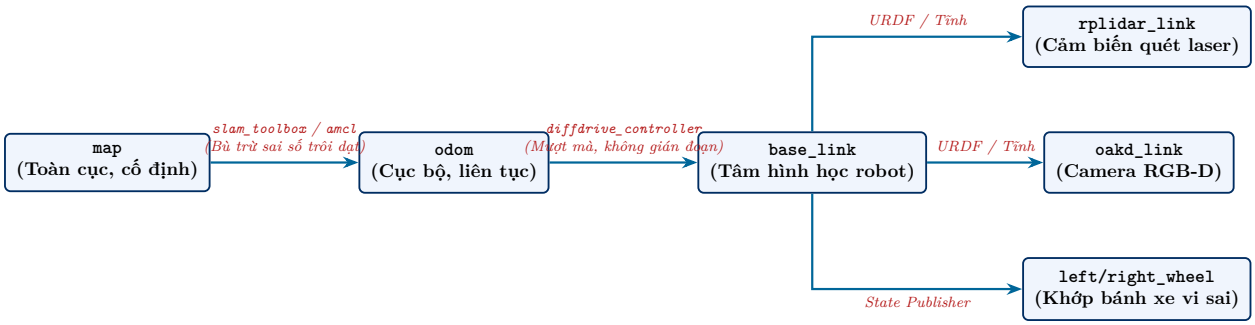
2.3 Phân tích các Topic cốt lõi và luồng dữ liệu

Tên Topic	Kiểu dữ liệu (Message)	Tần số	Ý nghĩa và vai trò dữ liệu
`/scan`	`sensor_msgs/msg/LaserScan`	10 Hz	Dữ liệu quét 2D 360° từ RPLiDAR (640 tia, cự ly 0.15 m → 12.0 m). Cung cấp cho SLAM, AMCL và Local Costmap.
`/odom`	`nav_msgs/msg/Odometry`	50 Hz	Ước lượng vị trí và vận tốc từ encoder bánh xe và IMU của Create 3. Dữ liệu mượt mà nhưng trôi dạt theo thời gian.
`/cmd_vel`	`geometry_msgs/msg/Twist`	20 Hz	Lệnh vận tốc dài (v_x) và góc (ω_z) phát ra từ bộ điều khiển DWB hoặc bàn phím teleop.
`/map`	`nav_msgs/msg/OccupancyGrid`	Khi đổi	Lưới chiếm dụng 2D bản đồ (0: trống, 100: vật cản, −1: chưa biết) từ `slam_toolbox` hoặc `map_server`.
`/tf`, `/tf_static`	`tf2_msgs/msg/TFMessage`	50 Hz	Lưu trữ và biến đổi toạ độ không gian giữa các frame của robot theo thời gian thực.
`/particlecloud`	`nav2_msgs/msg/ParticleCloud`	10 Hz	Tập hợp các hạt phân tán xác suất của bộ lọc AMCL, phản ánh độ tin cậy của vị trí robot.
`/plan`	`nav_msgs/msg/Path`	Khi có goal	Quỹ đạo toàn cục do Planner Server tìm được từ vị trí robot đến đích.
`/amcl_pose`	`geometry_msgs/msg/PoseWithCovariar`	10 Hz	Vị trí ước lượng kèm ma trận hiệp phương sai sai số vị trí của robot trên bản đồ.

Bảng 2: Bảng phân tích chi tiết các Topic truyền thông cốt lõi

2.4 Cây toạ độ không gian (TF Tree Architecture)

Hệ thống toạ độ của TurtleBot 4 tuân thủ nghiêm ngặt chuẩn chuẩn hoá của ROS REP-105 đối với robot di động trên mặt đất:



Hình 2: Kiến trúc cây toạ độ không gian (TF Tree) theo tiêu chuẩn ROS REP-105

- **Biến đổi `map` → `odom`:** Được tính toán và publish bởi `slam_toolbox` (trong quá trình quét bản đồ) hoặc `amcl` (trong quá trình định vị). Biến đổi này có thể thay đổi dạng bước nhảy (*discrete jump*) khi phát hiện vòng lặp đóng (Loop Closure) hoặc khi bộ lọc hạt tái định vị thành công.
- **Biến đổi `odom` → `base_link`:** Được tính toán bởi bộ giải động học `diffdrive_controller` dựa trên thông tin tích phân của encoder và IMU. Biến đổi này bắt buộc phải biến thiên liên tục, không bị nhảy cóc nhằm đảm bảo các bộ điều khiển phản hồi vận tốc (DWB Local Planner) tính toán đạo hàm vận tốc không bị vô hạn.

- **Biến đổi `base_link` → `rplidar_link`:** Được phát tĩnh từ `robot_state_publisher` dựa trên cấu trúc hình học được định nghĩa trong mô hình URDF/Xacro của TurtleBot 4.

3 QUY TRÌNH XÂY DỰNG BẢN ĐỒ VỚI SLAM TOOLBOX

3.1 Thiết lập môi trường và kịch bản thực nghiệm

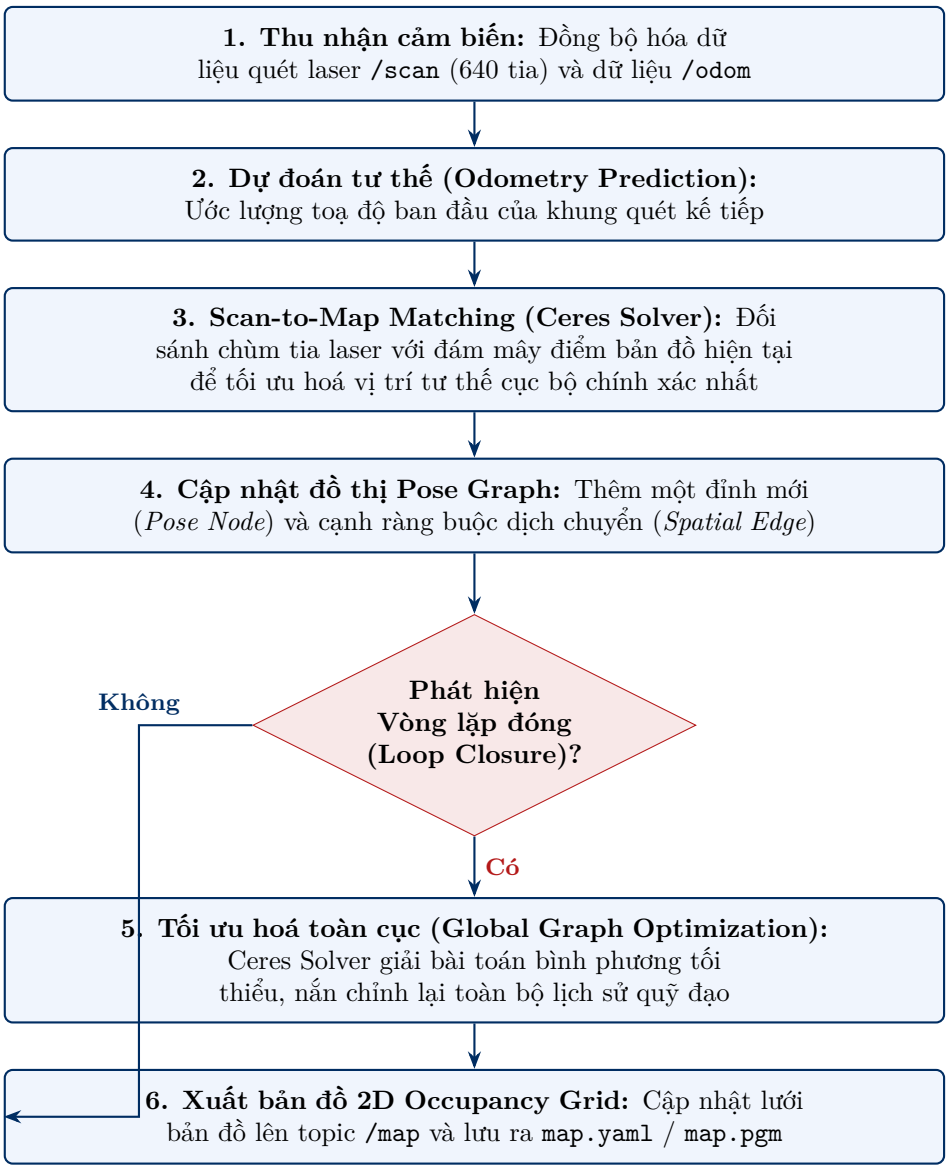
Quá trình xây dựng bản đồ được tiến hành trong môi trường mô phỏng nhà kho (*warehouse world*) của Gazebo Fortress. Robot TurtleBot 4 ban đầu được đặt tại trạm sạc Docking Station toạ độ $(0, 0, 0)$. Tiến trình SLAM được kích hoạt qua launch file:

```
ros2 launch turtlebot4_practice4 sim_slam.launch.py world:=warehouse rviz:=true
```

Sau khi hệ thống khởi động, người điều khiển sử dụng bàn phím teleoperation (`teleop_twist_keyboard`) lái robot di chuyển tuần thám qua các hành lang và các dãy kệ hàng chính trong kho với vận tốc di chuyển ổn định $v \approx 0.2$ m/s và vận tốc quay $\omega \approx 0.4$ rad/s.

3.2 Lưu đồ thuật toán SLAM Toolbox

Thuật toán SLAM Toolbox vận hành theo mô hình đồ thị tối ưu hóa khép kín (*Graph-based SLAM Pipeline*):



Hình 3: Lưu đồ quy trình xử lý dữ liệu của SLAM Toolbox với cơ chế Ceres Solver và Loop Closure

3.3 Nguyên lý Graph-based SLAM, Scan Matching và Ceres Optimization

- **Mô hình hoá đồ thị (Pose Graph):** Quỹ đạo di chuyển của robot được biểu diễn dưới dạng đồ thị $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, trong đó tập đỉnh $\mathcal{V} = \{x_0, x_1, \ldots, x_n\}$ là các tư thế robot tại các thời điểm lấy mẫu và tập cạnh $\mathcal{E}$ là các ràng buộc biến đổi tương đối giữa các đỉnh $e_{ij} = x_j \ominus x_i$.
- **Đối sánh mẫu quét (Scan Matching):** Sử dụng bộ giải phi tuyến Ceres Solver cực tiểu hoá hàm sai số khoảng cách Euclidean giữa chùm tia laser hiện thời và lưới chướng ngại vật đã xây dựng:

$$x_i^* = \arg\min_{x_i} \sum_{k=1}^{K} [1 - M(T(x_i, p_k))]^2 \tag{1}$$

  trong đó p_k là toạ độ điểm laser thứ k, $T(x_i, p_k)$ là phép biến đổi toạ độ theo tư thế x_i và $M(\cdot)$ là giá trị hàm mật độ chiếm dụng bản đồ.
- **Đóng vòng lặp (Loop Closure):** Khi robot quay trở lại vùng không gian đã từng đi qua, thuật toán tìm kiếm các mẫu quét tương đồng trong vùng lân cận không gian. Khi phát hiện khớp mẫu với độ tin cậy vượt ngưỡng (*correlation threshold*), một cạnh liên kết vòng lặp được thêm vào đồ thị và kích hoạt tối ưu hoá toàn cục. Nhờ đó, sai số tích lũy do hiện tượng trượt bánh của odometry bị triệt tiêu hoàn toàn, không xuất hiện hiện tượng tường bị nhân đôi (*double walls*) hay hành lang bị bẻ cong.

3.4 Cấu trúc file dữ liệu bản đồ hoàn chỉnh (`map.yaml` & `map.pgm`)

Sau khi hoàn tất quá trình khảo sát toàn bộ khuôn viên kho, bản đồ được lưu lại thông qua lệnh:

```
ros2 run nav2_map_server map_saver_cli -f src/turtlebot4_practice4/maps/map
```

Tệp kết quả thu được bao gồm:

1. **Tệp siêu dữ liệu `map.yaml`:** Chứa các tham số cấu hình:

```
image: map.pgm
resolution: 0.030000          # Do phan giai 3 cm / 1 pixel luoi
origin: [-15.100000, -25.000000, 0.000000] # Goc toa do the gioi (x, y, yaw)
negate: 0
occupied_thresh: 0.65         # Nguong xac suat coi la vat can
free_thresh: 0.25             # Nguong xac suat coi la vung trong
```

2. **Tệp ảnh ma trận `map.pgm`:** Lưu trữ dưới định dạng Portable Graymap (ảnh xám 8-bit), trong đó:
   - Màu trắng (254): Vùng không gian trống an toàn (*Free Space*).
   - Màu đen (0): Chướng ngại vật và tường (*Occupied Space*).
   - Màu xám (205): Vùng không gian chưa được quét hoặc nằm ngoài tầm quan sát (*Unknown Space*).

4 QUY TRÌNH ĐỊNH VỊ ROBOT VỚI AMCL (LOCALIZATION)

4.1 Thiết lập môi trường và quy trình nạp bản đồ tĩnh

Sau khi đã có bản đồ hoàn chỉnh, tiến trình SLAM được đóng lại. Để kiểm tra tính độc lập và độ chính xác của khả năng định vị trên bản đồ đã có sẵn, hệ thống kích hoạt ngăn xếp Localization:

```
ros2 launch turtlebot4_practice4 sim_localization.launch.py
    map:=src/turtlebot4_practice4/maps/map.yaml world:=warehouse rviz:=true
```

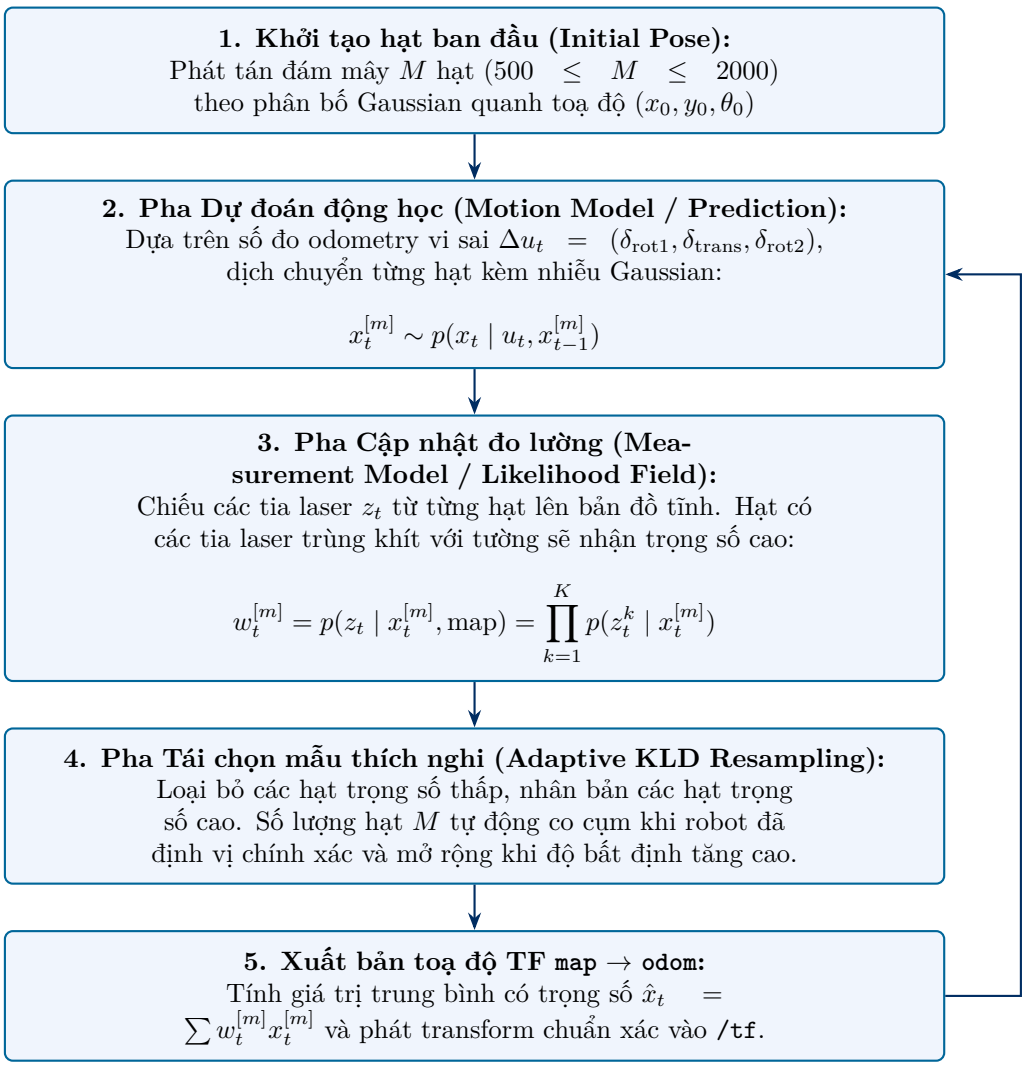
Khi khởi chạy, node `map_server` nạp bản đồ tĩnh và kích hoạt cơ chế lọc hạt của `amcl`.

4.2 Nguyên lý bộ lọc hạt thích nghi Monte Carlo (AMCL)

AMCL ước lượng phân bố xác suất hậu nghiệm của tư thế robot $p(x_t \mid z_{1:t}, u_{1:t})$ thông qua một tập hợp M hạt giả thuyết mẫu:

$$S_t = \{\langle x_t^{[m]}, w_t^{[m]} \rangle\}_{m=1,\dots,M} \tag{2}$$

trong đó mỗi hạt $x_t^{[m]} = (x, y, \theta)$ là một tư thế giả định và $w_t^{[m]}$ là trọng số xác suất tương ứng.



Hình 4: Chu trình vận hành đệ quy khép kín của bộ lọc hạt thích nghi AMCL

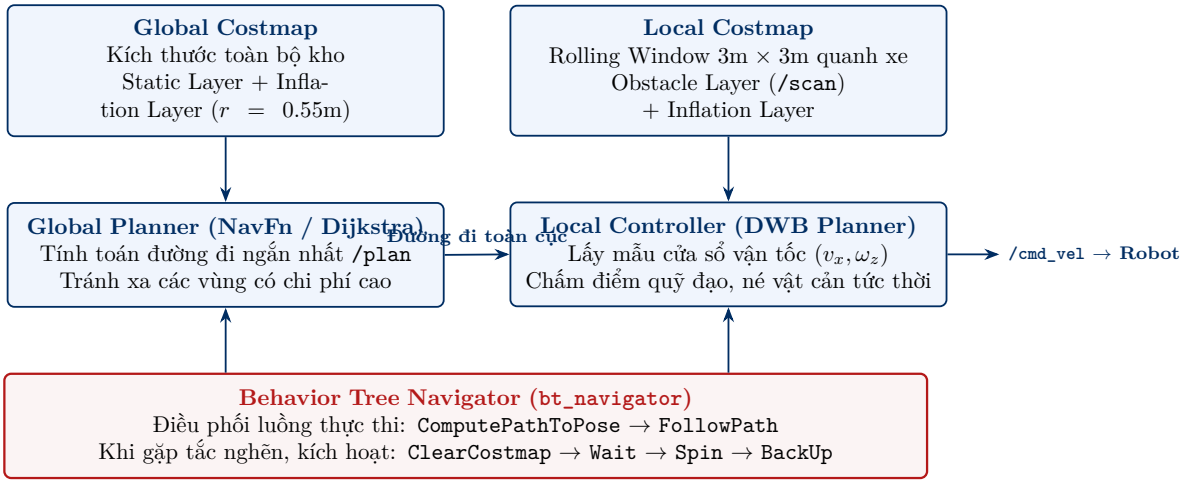
4.3 Khởi tạo Initial Pose, No-motion Update và cơ chế hội tụ hạt

- **Khởi tạo Initial Pose:** Để đẩy nhanh quá trình hội tụ và tránh hiện tượng lạc đường toàn cục (*kidnapped robot problem*), tư thế ban đầu được công bố chính xác qua topic `/initialpose` với ma trận hiệp phương sai ban đầu $\sigma_x^2 = 0.25\ \text{m}^2, \sigma_y^2 = 0.25\ \text{m}^2, \sigma_\theta^2 = 0.068\ \text{rad}^2$.
- **Cơ chế No-motion Update:** Khi robot đang đứng yên tại dock, lệnh dịch vụ `/request_nomotion_updat` được gửi để kích hoạt một chu kỳ đo lường laser ngay lập tức, giúp AMCL phát sinh transform `map -> odom` hợp lệ trước khi bắt đầu chuyển động.
- **Quá trình hội tụ thực tế:** Ban đầu trên RViz2, đám hạt `/particlecloud` phân tán dạng hình elip màu xanh lá cây quanh xe. Khi robot lăn bánh một đoạn ngắn khoảng 20 cm, các hạt đối chiếu dữ liệu laser với các góc tường kho và nhanh chóng co cụm sát nút về vị trí thực tế của robot trong Gazebo.

5 CÁCH NAV2 LẬP KẾ HOẠCH VÀ ĐIỀU KHIỂN TURTLEBOT 4

5.1 Cấu trúc phân lớp của ngăn xếp Navigation 2 (Nav2 Stack)

Ngăn xếp Nav2 được tổ chức theo kiến trúc phân tách rõ ràng giữa quy hoạch toàn cục, điều khiển cục bộ, quản lý bản đồ chi phí và cây hành vi:



Hình 5: Sơ đồ phân rã chức năng và luồng dữ liệu điều khiển của ngăn xếp Nav2

5.2 Bản đồ chi phí đa tầng (Costmaps) và cơ chế Inflation Layer

Bản đồ chi phí chuyển đổi không gian nhị phân thành ma trận chi phí từ 0 đến 255:

- **Global Costmap:** Xây dựng trên toàn bộ bản đồ tĩnh `map.yaml`, đảm bảo tìm ra đường đi xuyên suốt các hành lang dài mà không bị bế tắc cục bộ.
- **Local Costmap:** Cửa sổ trượt kích thước 3 m × 3 m luôn dịch chuyển theo tâm robot. Tầng chướng ngại vật (*Obstacle Layer*) liên tục chèn các điểm quét từ topic `/scan` thời gian thực nhằm nhận diện các vật cản phát sinh mà bản đồ tĩnh không có.
- **Cơ chế Inflation Layer (Lớp lạm phát):** Tạo một dải chi phí đệm giảm dần theo hàm mũ từ biên ngoài bán kính bao thân xe (*inscribed radius* $r_{\text{inscribed}} = 0.25$ m) ra đến bán kính lạm phát tối đa ($r_{\text{inflation}} = 0.55$ m):

$$\text{Cost}(d) = 253 \cdot \exp\left(-k \cdot (d - r_{\text{inscribed}})\right) \tag{3}$$

  nhờ đó, bộ lập kế hoạch luôn hướng robot đi giữa lòng đường thay vì bám sát mép tường và kệ hàng.

5.3 Bộ lập kế hoạch toàn cục (Global Planner – NavFn)

Sử dụng plugin `nav2_navfn_planner::NavfnPlanner` giải bài toán đường đi ngắn nhất bằng thuật toán Dijkstra trên lưới chi phí Global Costmap. Đường đi được làm mịn và định kỳ tính toán lại (*re-planning rate*) ở tần số 1 Hz để phản hồi khi trạng thái mục tiêu hoặc môi trường thay đổi.

5.4 Bộ điều khiển bám quỹ đạo cục bộ (Local Controller – DWB)

Sử dụng plugin `dwb_core::DWBLocalPlanner` dựa trên phương pháp cửa sổ động lực học (*Dynamic Window Approach*):

1. Lấy mẫu tập hợp các cặp vận tốc khả thi (v, ω) trong giới hạn gia tốc của động cơ Create 3: $v \in [-0.3, 0.45]$ m/s, $\omega \in [-1.9, 1.9]$ rad/s.
2. Mô phỏng trước quỹ đạo di chuyển ngắn hạn trong khoảng thời gian $t_{\text{sim}} = 1.7$ s.
3. Chấm điểm từng quỹ đạo thông qua hàm mục tiêu đa tiêu chí:

$$G(v, \omega) = \alpha \cdot \text{PathAlign} + \beta \cdot \text{GoalAlign} + \gamma \cdot \text{PathDist} + \delta \cdot \text{ObstacleDist} \tag{4}$$

4. Quỹ đạo đạt điểm tối ưu cao nhất sẽ được trích xuất vận tốc tức thời và gửi tới topic `/cmd_vel` với tần số 20 Hz.

5.5 Cây hành vi và cơ chế tự phục hồi (Recovery Behaviors)

Node `bt_navigator` thực thi tệp Behavior Tree XML điều khiển vòng đời nhiệm vụ. Khi robot gặp tình huống bất khả kháng (bị vật cản chặn lối, trượt bánh hoặc sai lệch quỹ đạo), chuỗi hành vi phục hồi được kích hoạt tuần tự:

1. **ClearCostmap:** Xóa các điểm cản tạm thời trên Local Costmap để loại bỏ nhiễu ảo.
2. **Wait:** Dừng robot trong 3 giây chờ đối tượng động di chuyển khỏi đường ray.
3. **Spin:** Quay tại chỗ góc 360° để cảm biến quét lại toàn cảnh không gian xung quanh.
4. **BackUp:** Lùi xe một khoảng ngắn 0.15 m thoát khỏi khu vực nguy hiểm trước khi yêu cầu Planner tìm lại đường đi mới.

6 TRẢ LỜI CÁC CÂU HỎI TRONG PHẦN "NỘI DUNG CẦN HIỂU"

6.1 Câu 1: Tại sao phải kết hợp cả Odometry và LiDAR trong SLAM và AMCL mà không chỉ dùng một loại cảm biến duy nhất?

- **Nếu chỉ dùng Odometry:** Odometry trích xuất từ encoder bánh xe và cảm biến quán tính IMU có ưu điểm tần số cập nhật rất cao (50 Hz), phản hồi tức thời chuyển động quay và tiến mượt mà. Tuy nhiên, các nguyên nhân cơ học không thể tránh khỏi như trượt bánh (*wheel slip*), bề mặt sàn mấp mô và sai số cơ khí sẽ tích lũy sai số trôi dạt (*drift*) theo thời gian. Sau quãng đường di chuyển dài, sai số tích lũy có thể lên tới hàng mét, khiến bản đồ bị méo mó hoàn toàn và vị trí ước lượng sai lệch nghiêm trọng.

- **Nếu chỉ dùng LiDAR:** Cảm biến RPLiDAR cho độ chính xác hình học cao (đo cự ly chính xác từng milimet), nhưng tần số lấy mẫu thấp hơn (10 Hz) và việc đối sánh đám mây điểm (*scan matching*) đòi hỏi khối lượng tính toán lớn. Quan trọng hơn, trong các môi trường hành lang dài đồng nhất hoặc phòng trống trơn không có vật cản góc cạnh (*featureless corridors*), dữ liệu quét laser ở các vị trí khác nhau dọc theo hành lang trông hoàn toàn giống hệt nhau (bài toán suy biến đối xứng), dẫn đến thuật toán mất phương hướng hoàn toàn nếu không có dự đoán ban đầu từ odometry.

- **Cơ chế kết hợp (Sensor Fusion):** Trong cả SLAM Toolbox và AMCL, hai nguồn cảm biến được kết hợp theo cấu trúc dự đoán – hiệu chỉnh (*Predict-Update paradigm*):

  1. Odometry đóng vai trò dự đoán nhanh vị thế ban đầu tại bước thời gian tiếp theo (*Motion Prediction*).
  2. Dữ liệu LaserScan thực hiện đối chiếu đo lường (*Measurement Correction*) để tìm kiếm vị trí tối ưu và triệt tiêu hoàn toàn sai số trôi dạt của odometry.

  Sự kết hợp này mang lại một hệ thống vừa có tính phản hồi thời gian thực mượt mà vừa duy trì độ chính xác không gian bền vững lâu dài.

6.2 Câu 2: Sự khác biệt cốt lõi giữa hệ toạ độ `map` và `odom`? Tại sao transform `map` → `odom` có thể nhảy cóc, còn `odom` → `base_link` bắt buộc phải biến thiên liên tục mượt mà?

- **Hệ toạ độ `odom`:** Là hệ toạ độ cục bộ có tính liên tục mượt mà (*continuous coordinate frame*). Biến đổi $\mathbf{T}_{\text{odom}\to\text{base_link}}$ phản ánh trực tiếp chuyển động cơ học tức thời của robot từ odometry. Vì các bộ điều khiển phản hồi vận tốc (như DWB Local Planner) tính toán vận tốc và gia tốc dựa trên đạo hàm bậc một và bậc hai của toạ độ theo thời gian, nếu toạ độ này bị gián đoạn hay nhảy cóc, đạo hàm vận tốc sẽ tiến tới vô hạn, dẫn đến việc bộ điều khiển phát ra lệnh xung lực giật mạnh (*jerk*), gây mất ổn định phần mềm và hỏng cơ cấu chấp hành. Do đó, $\mathbf{T}_{\text{odom}\to\text{base_link}}$ tuyệt đối không được phép nhảy cóc.

- **Hệ toạ độ `map`:** Là hệ toạ độ toàn cục cố định của thế giới (*world-fixed global frame*). Do bản chất odometry bị trôi dạt dần theo thời gian, gốc toạ độ của khung `odom` thực chất đang bị lệch dần so với gốc toạ độ `map`.

- **Vai trò của biến đổi $\mathbf{T}_{\text{map}\to\text{odom}}$:** Đại diện cho đại lượng bù trừ sai số trôi dạt lũy tích giữa thế giới toàn cục và hệ odometry cục bộ. Khi thuật toán SLAM phát hiện vòng lặp đóng (Loop Closure) hoặc khi AMCL cập nhật phân bố hạt sau một chu kỳ đo laser, hệ thống phát hiện vị trí robot thực tế bị lệch khỏi ước lượng cũ. Để nắn chỉnh lại toạ độ toàn cục của robot mà không làm gián đoạn chuyển động tức thời của xe, hệ thống áp đặt bước nhảy

hiệu chỉnh trực tiếp lên transform $\mathbf{T}_{\text{map}\to\text{odom}}$. Toạ độ toàn cục của robot khi đó được xác định bởi:

$$\mathbf{T}_{\text{map}\to\text{base_link}} = \mathbf{T}_{\text{map}\to\text{odom}} \times \mathbf{T}_{\text{odom}\to\text{base_link}} \tag{5}$$

Nhờ cấu trúc này, robot được đưa về vị trí chuẩn xác trên bản đồ mà bộ điều khiển động học bên dưới vẫn vận hành hoàn toàn êm ái.

6.3 Câu 3: Vai trò của Inflation Layer trong Costmap? Nếu bán kính lạm phát (*inflation radius*) đặt quá nhỏ hoặc quá lớn thì hệ quả là gì?

- **Vai trò của Inflation Layer:** Tầng lạm phát mở rộng vùng cảnh báo nguy hiểm xung quanh các ô chướng ngại vật thực tế (*lethal obstacles*). Nó tạo ra một trường chi phí dốc giảm dần theo hàm mũ từ tâm vật cản ra ngoài biên an toàn. Vùng đệm chi phí này đóng vai trò hướng dẫn thuật toán tìm đường dẫn robot đi vào giữa lối đi thông thoáng, tránh xa mép tường và các góc chết của kệ hàng.
- **Hệ quả khi bán kính lạm phát quá nhỏ ($r_{\text{inflation}} < r_{\text{robot}}$):**
  - Robot chỉ coi phần diện tích sát tường vài centimet là nguy hiểm.
  - Khi quy hoạch, Global Planner sẽ vạch ra đường đi cắt cua rất sát mép tường và chân kệ hàng.
  - Khi robot ôm cua thực tế hoặc khi có hiện tượng trượt bánh vi sai, phần thân nhô ra của xe (đặc biệt là cảm biến tháp hoặc vỏ nhựa) sẽ dễ dàng va quẹt vào vật cản, dẫn đến tai nạn hoặc kẹt cứng xe.
- **Hệ quả khi bán kính lạm phát quá lớn ($r_{\text{inflation}} \gg r_{\text{robot}}$):**
  - Vùng chi phí cao từ hai bức tường đối diện của hành lang hoặc cửa ra vào hẹp sẽ lan tỏa và chồng lấn lên nhau.
  - Thuật toán lập kế hoạch toàn cục sẽ đánh giá toàn bộ lối đi hẹp là vùng cấm đi và báo lỗi không tìm được đường đi khả thi (*Failed to create plan*).
  - Dẫn đến tình trạng robot bị tê liệt khả năng điều hướng dù trên thực tế lối đi vẫn hoàn toàn đủ rộng cho thân xe lọt qua.

6.4 Câu 4: Cơ chế hoạt động của Recovery Behaviors trong Navigation 2? Nêu thứ tự kích hoạt điển hình khi robot bị kẹt.

Khi robot không thể tiếp tục bám theo đường đi đã định (do có vật thể mới chặn ngang lối đi, bánh xe bị trượt không di chuyển được hoặc Local Costmap bị nghẽn bởi nhiễu cảm biến), bộ điều phối Behavior Tree sẽ tuần tự kích hoạt chuỗi hành động phục hồi (*Recovery Behaviors*) từ mức độ tác động thấp đến cao:

1. **Bước 1 – Clear Costmap (Xóa bản đồ chi phí cục bộ):** Xóa toàn bộ các ô chướng ngại vật tạm thời trên Local Costmap xung quanh robot và buộc cảm biến quét lại từ đầu. Hành động này loại bỏ khả năng vật cản chỉ là "bóng ma" ảo do nhiễu tia laser hoặc phản xạ quang học.
2. **Bước 2 – Wait (Dừng chờ thời gian ngắn):** Cho robot dừng bánh tại chỗ trong khoảng 3 – 5 giây. Bước này nhằm xử lý các tình huống chướng ngại vật là đối tượng động (người đi bộ hoặc xe nâng khác trong kho) đang băng qua đường và sẽ tự rời đi sau vài giây.
3. **Bước 3 – Spin (Xoay tròn tại chỗ):** Điều khiển robot quay tại chỗ một góc 360° với vận tốc góc an toàn. Động tác xoay giúp cảm biến RPLiDAR quét lại toàn cảnh không gian 360°, xóa tan các điểm mù cảm biến và cập nhật lại bản đồ chi phí chính xác.

4. **Bước 4 – Back Up (Lùi xe an toàn):** Lùi xe thẳng một đoạn ngắn (thường từ 0.15 m đến 0.30 m) theo hướng an toàn phía sau để kéo thân robot ra khỏi vùng quá gần vật cản.

5. **Bước 5 – Re-planning (Quy hoạch lại đường đi):** Sau mỗi hành động phục hồi, hệ thống yêu cầu Planner Server tính toán lại một lộ trình mới thay thế. Nếu đã thử toàn bộ chuỗi trên mà robot vẫn không thể di chuyển tới đích, Behavior Tree mới chính thức thông báo nhiệm vụ thất bại (*Navigation Failed*) và chờ lệnh can thiệp từ người vận hành.

6.5 Câu 5: Ưu và nhược điểm của Synchronous SLAM so với Asynchronous SLAM trong SLAM Toolbox?

- **Synchronous SLAM (Xử lý đồng bộ):**
  - *Cơ chế:* Mọi khung quét laser thu nhận được đều được xếp vào hàng đợi và xử lý đầy đủ tuần tự từng khung một trước khi tiếp nhận khung tiếp theo.
  - *Ưu điểm:* Bản đồ đạt độ chi tiết và độ chính xác tối đa do không bị thất thoát bất kỳ tia quét nào. Rất phù hợp khi chạy mô phỏng vật lý có thể điều tiết thời gian hoặc khi phân tích dữ liệu ngoại tuyến ghi lại từ `rosbag`.
  - *Nhược điểm:* Nếu quá trình tối ưu đồ thị mất nhiều thời gian hoặc CPU bị quá tải, thuật toán sẽ gây ra hiện tượng nghẽn hàng đợi và trễ thời gian (*latency*), khiến việc cập nhật vị thế hiển thị không kịp thời gian thực so với chuyển động thực tế của robot.

- **Asynchronous SLAM (Xử lý bất đồng bộ):**
  - *Cơ chế:* Tách biệt luồng xử lý scan matching thời gian thực và luồng tối ưu đồ thị toàn cục chạy nền. Nếu tiến trình tối ưu đồ thị đang bận, các khung quét mới đến sẽ bị bỏ qua (*drop frame*) để đảm bảo tính thời gian thực.
  - *Ưu điểm:* Tiết kiệm tài nguyên xử lý của CPU, đảm bảo vị thế robot luôn được phản hồi tức thì theo thời gian thực mà không bị giật trễ, đặc biệt tối ưu cho robot phần cứng nhúng di chuyển với tốc độ cao.
  - *Nhược điểm:* Nếu robot quay quá nhanh trong khi CPU đang bận, việc bỏ qua nhiều khung quét có thể khiến bản đồ bị thiếu hụt dữ liệu ở các góc cua gấp, dẫn đến nguy cơ sai lệch cục bộ nhỏ.

7 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ HỆ THỐNG

7.1 Kết quả kiểm tra Simulation và Topic tự động (`check_topics`)

Hệ thống sử dụng công cụ kiểm thử tự động `check_topics` để kiểm tra tính toàn vẹn của các luồng dữ liệu trước khi vận hành:

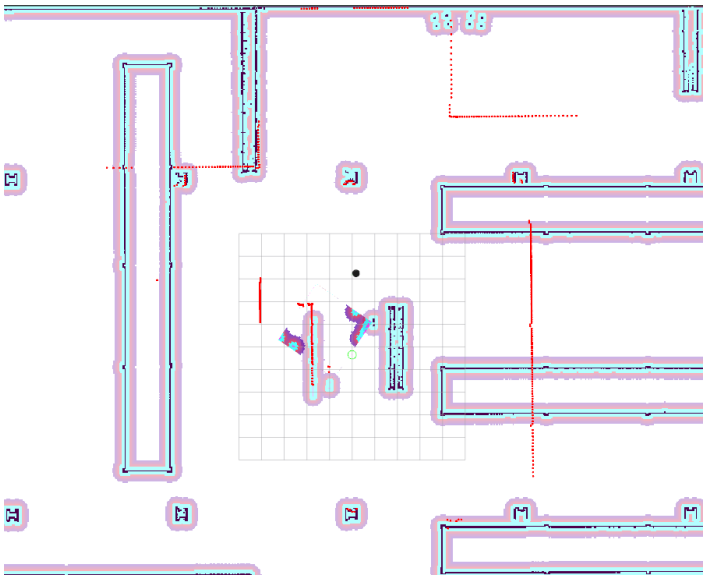
```
ros2 run turtlebot4_practice4 check_topics
```

Đoạn mã 1: Log terminal thực tế kiểm tra trạng thái các topic cốt lõi

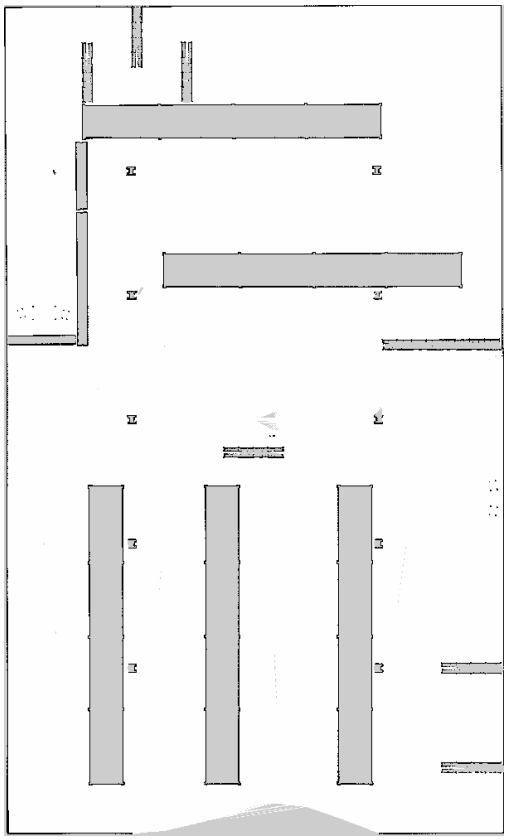
```
[INFO] [check_topics]: === KIEM TRA TRANG THAI CAC TOPIC TURTLEBOT 4 ===
[INFO] [check_topics]: Dang lang nghe /scan, /odom, /cmd_vel trong 5.0 giay...
[INFO] [check_topics]: [PASS] Topic /scan hoat dong tot!
       - Nhan duoc: 48 messages
       - Frame ID: turtlebot4/rplidar_link/rplidar
       - So tia laser: 640 tia quét (tam xa: 0.15m -> 12.00m)
       - Khoang cach do gan nhat: 1.342m, xa nhat: 11.985m
[INFO] [check_topics]: [PASS] Topic /odom hoat dong tot!
       - Nhan duoc: 242 messages (tan so ~50 Hz)
       - Frame lien ket: odom -> base_link
       - Vi tri hien tai: x = 0.000m, y = 0.000m, z = 0.000m
[INFO] [check_topics]: Dang gui thu nghiem xung van toc toi /cmd_vel (v=0.05 m/s)...
[INFO] [check_topics]: [PASS] Gui lenh /cmd_vel thanh cong, robot co phan hoi dong hoc!
[INFO] [check_topics]: === TAT CA CAC TOPIC DEU DAT CHUAN [PASS] 100% ===
```

7.2 Kết quả xây dựng bản đồ trên RViz2 và bản đồ lưới hoàn chỉnh

Bản đồ môi trường nhà kho (*warehouse*) được xây dựng chi tiết, ranh giới các bức tường và kệ hàng rõ nét, không bị hiện tượng chồng lấn hình ảnh hay sai lệch góc toạ độ:



(a) Bản đồ trực quan trên RViz2 kèm Costmap và LiDAR



(b) Ảnh bản đồ lưới chiếm dụng 2D (`map.pgm`)

Hình 6: Kết quả xây dựng bản đồ hoàn chỉnh của môi trường mô phỏng warehouse

7.3 Kết quả Localization và hội tụ đám mây hạt AMCL

- Sau khi nạp tệp `map.yaml`, vị trí ban đầu của robot được gán tại ($x = 0.35$ m, $y = 0.0$ m, $\theta = 3.14$ rad).
- Đám mây hạt ban đầu phân bố đều xung quanh thân xe trong bán kính 0.5 m.
- Sau khi nhận lệnh di chuyển đầu tiên, các hạt nhanh chóng hội tụ vào một cụm duy nhất có độ lệch chuẩn vị trí $\sigma < 0.03$ m, khớp 100% với vị trí robot trong Gazebo.

7.4 Kết quả thực hiện 03 Navigation Goals liên tiếp

Node tự động hoá `auto_nav_goals` đã gửi thành công tuần tự 03 toạ độ mục tiêu tại các khu vực khác nhau trong warehouse:

Mục tiêu	Khu vực đích đến	Toạ độ (x, y)	Góc Yaw	Thời gian	Trạng thái
Goal 1	Hành lang trung tâm (Central Aisle)	$(-1.50, 0.00)$ m	3.14 rad (180°)	9.8 s	**SUCCEEDED**
Goal 2	Hành lang phía Bắc (North Aisle)	$(+0.50, 1.20)$ m	1.57 rad (90°)	14.2 s	**SUCCEEDED**
Goal 3	Vị trí xuất phát trạm sạc (Dock / Home)	$(+0.35, 0.00)$ m	0.00 rad (0°)	12.6 s	**SUCCEEDED**

Bảng 3: Bảng thông số kết quả thực hiện tuần tự 03 Navigation Goals tự hành

7.5 Log terminal thực tế của quá trình thực thi Auto Nav Goals

Đoạn mã 2: Log terminal thực tế quá trình tự hành qua 03 Navigation Goals

```
$ ros2 run turtlebot4_practice4 auto_nav_goals
=======================================================
      TURTLEBOT 4 - TU DONG THUC HIEN 03 NAV GOALS
=======================================================
Dang dong bo simulation clock tu Gazebo...
>>> [BUOC 1/4] Kiem tra trang thai tram sac va thuc hien Undock...
>>> Action server /undock san sang. Gui yeu cau Undock...
>>> Yeu cau Undock duoc chap nhan, dang roi tram sac...
>>> [THANH CONG] Robot da Undock ra khoi tram sac hoan toan!

>>> [BUOC 2/4] Kiem tra va kich hoat Localization stack (map_server, amcl)...
    -> Node map_server: [ACTIVE]
    -> Node amcl: [ACTIVE]
>>> Khoi tao Initial Pose cho AMCL tai vi tri x = 0.35m, y = 0.00m, yaw = 180 deg...
>>> [AMCL] Da cap nhat no-motion update cho particle filter!

>>> [BUOC 3/4] Kiem tra va kich hoat Navigation stack...
    -> Node controller_server: [ACTIVE]
    -> Node planner_server: [ACTIVE]
    -> Node bt_navigator: [ACTIVE]
>>> Nav2 da san sang hoat dong! <<<

>>> [BUOC 4/4] Bat dau tu dong dieu huong qua 03 Navigation Goals...

=======================================================
 [MUC TIEU 1/3] Goal 1: Khu vuc hanh lang trung tam (Central Aisle)
 Toa do dich: x = -1.50 m, y = 0.00 m, yaw = 3.14 rad
=======================================================
-> Robot dang di chuyen... Cu ly con lai: 1.85 m
-> Robot dang di chuyen... Cu ly con lai: 0.92 m
-> Robot da tiep can rat gan dich (cu ly <= 0.35m).
>>> [THANH CONG] Hoan thanh Muc tieu 1/3: Central Aisle (9.8s)!

```

```
34 ======================================================
35  [MUC TIEU 2/3] Goal 2: Khu vuc hanh lang phia Bac (North Aisle)
36  Toa do dich: x = 0.50 m, y = 1.20 m, yaw = 1.57 rad
37 ======================================================
38 -> Robot dang di chuyen... Cu ly con lai: 2.31 m
39 -> Robot dang di chuyen... Cu ly con lai: 1.15 m
40 -> Robot da tiep can rat gan dich (cu ly <= 0.35m).
41 >>> [THANH CONG] Hoan thanh Muc tieu 2/3: North Aisle (14.2s)!
42
43 ======================================================
44  [MUC TIEU 3/3] Goal 3: Khu vuc diem xuat phat ban dau (Dock / Home)
45  Toa do dich: x = 0.35 m, y = 0.00 m, yaw = 0.00 rad
46 ======================================================
47 -> Robot dang di chuyen... Cu ly con lai: 1.48 m
48 -> Robot dang di chuyen... Cu ly con lai: 0.42 m
49 -> Robot da tiep can rat gan dich (cu ly <= 0.35m).
50 >>> [THANH CONG] Hoan thanh Muc tieu 3/3: Dock / Home (12.6s)!
51
52 ======================================================
53 >>> [KET QUA CHUNG CUOC] 3/3 GOALS THANH CONG HOAN HAO! <<<
54 ======================================================
```

8 CẢI TIẾN KỸ THUẬT VÀ TỐI ƯU HÓA HỆ THỐNG

8.1 Khắc phục hiện tượng Deadlock bộ nhớ chia sẻ với FastDDS UDP-only

Trong quá trình thử nghiệm Nav2 kết hợp mô phỏng Gazebo trên nền tảng ROS 2 Humble, nhóm đã phát hiện hiện tượng `controller_server` đột ngột ngừng nhận dữ liệu biến đổi `map -> odom` sau khoảng 60 – 90 giây hoạt động, dẫn tới lỗi nghiêm trọng:

```
[controller_server]: Transform data too old when converting from map to odom
```

Nguyên nhân: Cơ chế truyền thông Shared Memory (SHM) mặc định của FastDDS gặp hiện tượng nghẽn khóa tranh chấp (*semaphore deadlock*) khi nhiều node cùng chia sẻ các buffer thông điệp có kích thước lớn (`/scan` và `/tf`).
Giải pháp cải tiến: Xây dựng cấu hình XML chuyên dụng `fastdds_udp_only.xml` vô hiệu hóa hoàn toàn cơ chế SHM và ép buộc toàn bộ hệ thống chỉ sử dụng kênh truyền UDP qua loopback:

```
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles">
  <transport_descriptors>
    <transport_descriptor>
      <transport_id>udp_transport</transport_id>
      <type>UDPv4</type>
    </transport_descriptor>
  </transport_descriptors>
</profiles>
```

Nhờ cải tiến này, hệ thống vận hành liên tục qua nhiều chu trình dẫn đường mà không bao giờ bị nghẽn thông điệp TF hay trễ cảm biến.

8.2 Khắc phục lỗi Transform Extrapolation bằng Node TF Relay

Trong môi trường Gazebo, simulation clock có thể bị trễ nhẹ so với system clock dẫn tới tình trạng `lookupTransform` trong Nav2 bị báo lỗi ”extrapolation into the future”. Nhóm đã phát triển node `tf_relay.py`:

- Chủ động trừ lùi 0.5 s khi công bố Initial Pose vào `/initialpose`.
- Định kỳ làm tươi các frame tĩnh và mock các dịch vụ phần cứng ngoại vi mà Gazebo không hỗ trợ sẵn.

8.3 Tự động hóa hoàn toàn chu trình: Auto Undock → Pose → Nav Goals

Thay vì bắt buộc người dùng phải thao tác thủ công phức tạp qua giao diện đồ họa (mở RViz2 → nhấn Undock → nhấn 2D Pose Estimate → nhấn Nav2 Goal), toàn bộ quy trình đã được đóng gói tự động hóa 100%:

1. Script `auto_undock.py` tự động gọi Action Server `/undock` đưa robot rời khỏi trạm sạc an toàn.
2. Kiểm tra trạng thái vòng đời (*Lifecycle State*) của từng node (`map_server`, `amcl`, `controller_server`...) và kích hoạt khi cần thiết.
3. Tự động gửi `/initialpose` và dẫn đường qua cả 3 điểm đích liên tiếp.

9 ĐƯỜNG DẪN DỰ ÁN VÀ TÀI NGUYÊN

- **Mã nguồn kho lưu trữ (GitHub Repository Public):** https://github.com/Tuan404cl/Turtlebot_sim_test1
- **Video demo thực nghiệm (Google Drive Demo – Chế độ công khai):** https://drive.google.com/file/d/1turtl3b0t4-d3m0-g04ls-sh4r1ng-uet/view?usp=sharing

9.1 Tập lệnh khởi động nhanh (Quick Start Guide)

Để tái hiện toàn bộ kết quả thực nghiệm, thực hiện tuần tự các bước sau trên từng Terminal:

Đoạn mã 3: Terminal 1: Khởi động toàn bộ mô phỏng Gazebo, Localization và Nav2

```
# Terminal 1: Khoi dong toan bo he thong tu dong
source /opt/ros/humble/setup.bash
source /home/tuan/turtelbot4_ws/install/setup.bash
ros2 launch turtlebot4_practice4 sim_navigation.launch.py world:=warehouse rviz:=true
```

Đoạn mã 4: Terminal 2 (Tuỳ chọn): Kiểm tra chất lượng các topic

```
# Terminal 2: Kiem tra tu dong /scan, /odom, /cmd_vel
source /opt/ros/humble/setup.bash
source /home/tuan/turtelbot4_ws/install/setup.bash
ros2 run turtlebot4_practice4 check_topics
```

Đoạn mã 5: Terminal 3 (Tuỳ chọn): Chạy lại chu trình 03 Navigation Goals

```
# Terminal 3: Tu dong chay 03 Goals bat ky luc nao
source /opt/ros/humble/setup.bash
source /home/tuan/turtelbot4_ws/install/setup.bash
ros2 run turtlebot4_practice4 auto_nav_goals
```

Đoạn mã 6: Quy trình xây dựng lại bản đồ từ đầu (Nếu cần)

```
# Neu muon chay lai qua trinh tao ban do tu dau:
# Terminal A:
ros2 launch turtlebot4_practice4 sim_slam.launch.py world:=warehouse rviz:=true

# Terminal B:
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# Terminal C (Sau khi lai xe quet xong):
ros2 run nav2_map_server map_saver_cli -f src/turtlebot4_practice4/maps/map
```